Temporal

What does an agent need to remember?

エージェントは何を覚えておく必要があるのか？

It can remember the conversation and forget the work.

Melanie Warrick · AI & DevRel @ Temporal



WHAT JUST HAPPENED

A memory store could bring back the answers. Could it bring back the work?

メモリストアは答えを取り戻せる。でも、作業そのものは？

> It doesn’t know if the refund went out.
>
> It couldn’t tell which step runs next.
>
> Nothing would restart the run.

Stripe knew the payment went through. Nothing kept track of where the work stood.

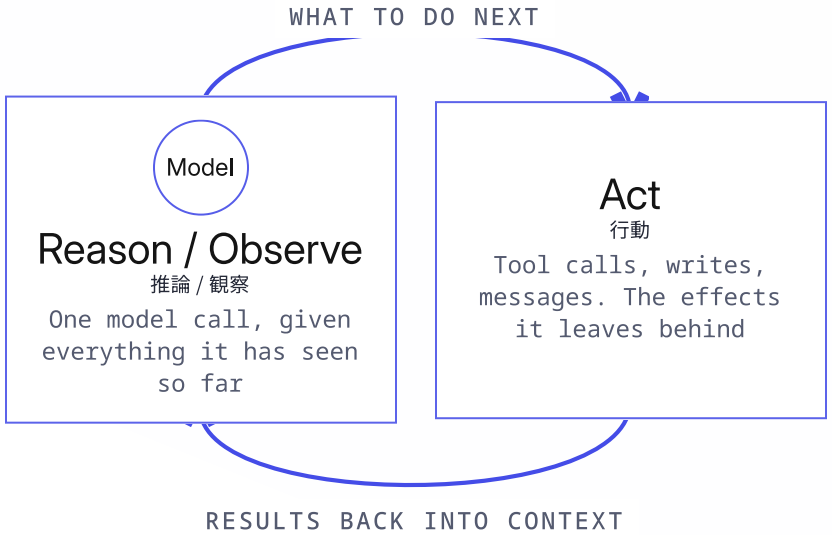


THE LOOP

An autonomous agent is a loop

自律型エージェントはループである

It is literally a loop. Plain Python, no SDK, no magic.



The result is just context for the next model call. It repeats until it reaches a goal, a stop condition, or a failure.

WHERE AGENTS SIT

AI agency levels

AIの自律性レベル

Level 1

Programmatic

プログラム制御

Control flow entirely in code.

Easy to reason about program behavior.



Level 2

Guided

ガイド付き

Control flow mostly in code. LLM makes limited decisions.

May require human review.



Level 3

Agentic

エージェント型

The LLM makes high-level control flow decisions.

Observability and guardrails are critical.



Level 4

Code-Gen

コード生成

The LLM actively writes and executes code.

Requires sandboxing and other controls.



Level 5

Self-Evolving

自己進化

The agent harness can modify its own code.

Needs bulletproof isolation and audit.



Level 3+: path chosen at runtime → can't pre-declare states

AROUND THE LOOP

Only context lives inside the loop

ループの中にあるのはコンテキストだけ

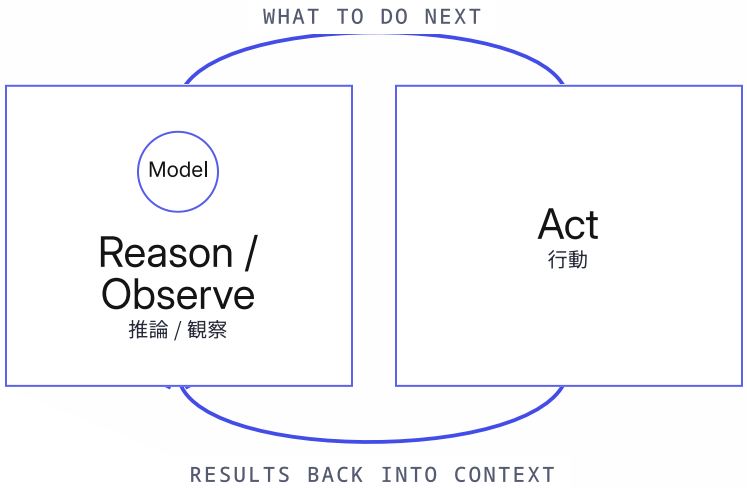
WHAT THE AGENT KNOWS WHERE THE WORK STANDS

IN THE LOOP

CONTEXT

コンテキスト

The messages list, rebuilt every turn



OUTSIDE THE LOOP

LONG-TERM MEMORY

長期メモリ

What carries across runs, in a store

OUTSIDE THE LOOP

STATE

状態

Facts the model does not own

The loop holds context. Who holds memory and state?

ループが持つのはコンテキスト。メモリと状態は誰が持つ？

CONCRETELY

What goes where

何をどこに置くか

One autonomous support agent, with authority to issue a refund.

CONTEXT コンテキスト

What the model sees this turn

- The customer's message
- The order it just looked up
- The last few tool results

LONG-TERM MEMORY 長期メモリ

What it carries across runs

- The refund policy
- This customer's past cases
- What went wrong last time

STATE 状態

Facts the model does not own

- Which tools already ran
- The refund it issued
- Blocked waiting on approval

Lose the first two and the agent repeats work. Lose the third and it refunds the same order twice.



THE FAMILIAR TAXONOMY

Memory comes in kinds

メモリにはいくつかの種類がある

WORKING
作業記憶

Like a whiteboard.

What is active for the current decision.

`The messages sent to the model`

EPISODIC
エピソード記憶

Like a diary.

Past experiences and events.

`Run logs, transcripts`

SEMANTIC
意味記憶

Like an encyclopedia.

Facts as the agent recalls them.

`Vector store, knowledge base`

PROCEDURAL
手続き記憶

Like a recipe.

Skills, and how to do things.

`Skill files, tool definitions`

PARAMETRIC
パラメトリック記憶

Like what you already knew.

Baked into the weights · can't edit while running

All of it answers one question: what should the agent know?

すべては一つの問いに行き着く：エージェントは何を知っておくべきか？

CONTEXT MANAGEMENT

Managing what the model sees

モデルに見せる情報を管理する

The window is a buffer. Context prep is what makes it behave like working memory.

01	Stuff the window すべて詰め込む	Everything in the prompt
02	Truncate 古い分を切り捨て	Drop the oldest turns
TODAY'S DEFAULT 03	Compact 要約して圧縮	Summarize old turns
04	Retrieve 必要な分を検索	RAG, memory files, memory tools
SHIPPED 05	Bigger windows より大きなウィンドウ	A million tokens, already here
EMERGING 06	Self-curating context 自己編集するコンテキスト	The agent edits its own playbook

Long-running loop → context grows every turn → so does the history

Manages what it knows → not where the work stands

PARTS OF THE SYSTEM

Facts have owners. Each owner is a system of record

事実には持ち主がいる。それぞれが正となる記録システム

Temporal owns one lane → read the rest live

TEMPORAL'S LANE

Execution state

実行状態

`Temporal, in this demo`

- Which steps already ran
- The attempt, and its identity
- Where the loop is blocked

Authorization state

認可状態

`Your auth system`

- May this agent act at all
- Scope of what it may touch
- Revoked since it started

Effect state

副作用の状態

`Stripe, your APIs`

- Did the charge clear
- Did the refund actually post
- Attempted, or committed

Domain state

ドメイン状態

`Your database`

- The refund row
- The customer's plan
- The ticket's status

The test: does another system already own this fact?

判断基準：その事実は、すでに別のシステムが持っていないか？

WORKED EXAMPLE

No single system knows the whole refund

返金の全体を知っているシステムはひとつもない

`Order 1234, at a single moment in the run.`

WHAT THE AGENT SEES Context コンテキスト	“Refund order 1234 for $80.”
WHAT THE AGENT REMEMBERS Long-term memory 長期メモリ	“Loyal customer. We refunded them once before.” `Useful, but it may be out of date.`
WHAT HAPPENED Effect state 副作用の状態	Stripe: Refund succeeded.
WHERE THE WORK STANDS Execution state 実行状態	Temporal: Refund step still in progress.

First two → decide · last two → recover safely

Disagree → the record wins, or you refund twice

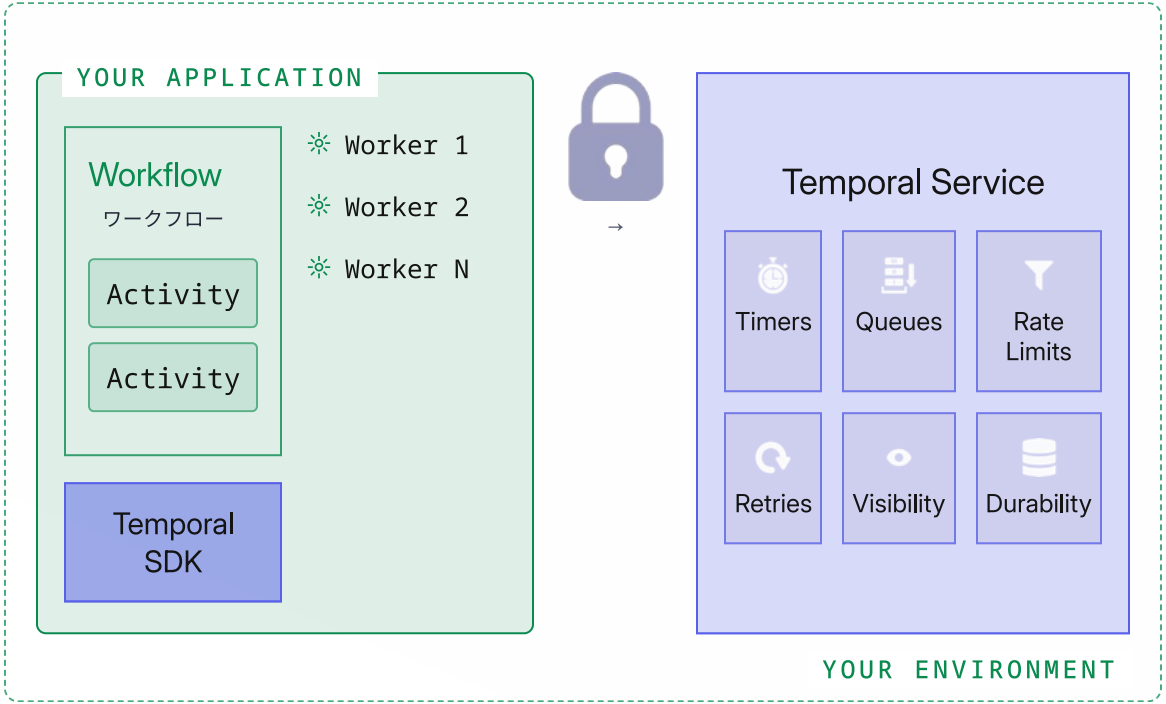
食い違ったら記録が優先。そうしないと二重返金になる

THE ENGINE UNDERNEATH

What is Temporal?

Temporalとは？

- Build crash-proof applications
- Open source, MIT license
- Write in regular code: Python, TypeScript, Java, Go, Rust, Ruby, .NET
- Uses *memoization* and *replay* to recover from failures at any point



TEMPORAL · THE VOCABULARY

Core Temporal concepts

Temporalの基本概念

WORKFLOW

ワークフロー

your function · the Temporal Service records its steps

`RefundApprovalAgent`

ACTIVITY

アクティビティ

One call that leaves the process. Retried on failure.

`agent_decide_next_step, issue_refund`

WORKER

ワーカー

The process that runs your code. Disposable.

`your Python process`

TASK QUEUE

タスクキュー

How work reaches a Worker. Workers poll it.

`refund-stage-<id>`

EVENT HISTORY

イベント履歴

The record of what already finished.

`order looked up, policy checked`

You write Workflows and Activities. Temporal keeps the history, and Workers do the work.

THE SHAPE IN A REAL SYSTEM

Where the durable loop runs

永続的なループはどこで動くのか

01

Client

クライアント

Starts the run and hands it off.

EVENT HISTORY

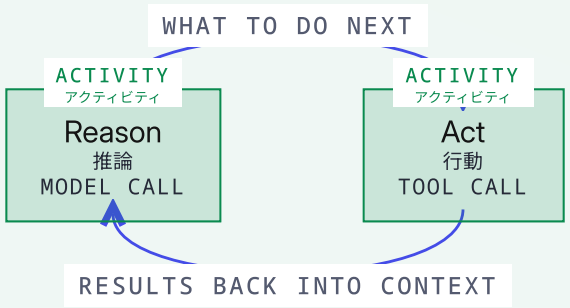
イベント履歴

stored by the Temporal Service

each step recorded

02 WORKER · THE PROCESS YOU RUN ワーカー

03 WORKFLOW · KEEPS THE ORDER ワークフロー



Worker stops → Workflow stays Running, keeps loop + state

ワーカーが止まっても、ワークフローは実行中のまま、ループと状態を保持する

The loop, in Python Pythonで書いたループ

Tools: lookup_order, lookup_customer_history, check_refund_policy, issue_refund. One of them moves money.

```
def run(request):
    working_memory = []                # all it knows, in-process
    decision = None
    for turn in range(MAX_TURNS):
        step = agent_decide_next_step(request, working_memory)  # REASON
        if step.action == "decide":
            decision = step
            break
        result = run_tool(step.tool, request)          # ACT
        working_memory.append({"tool": step.tool, "result": result})
    if not decision.recommendation == "approve":
        return escalate(decision)
    return issue_refund(request, decision)             # no idempotency
```

No SDK, no framework. Everything the agent knows lives in the `working_memory` list. Nothing outside the process knows how far it got.

The same loop, durable 同じループを永続化する

The same loop, same names, same order.

```python
@workflow.defn(name="RefundApprovalAgent")
class RefundWorkflow:
    @workflow.run
    async def run(self, request):
        for turn in range(MAX_TURNS):
            # REASON
            step = await workflow.execute_activity(agent_decide_next_step, ...)
            if step.action == "decide":
                decision = RefundDecision(...); break
            # ACT
            result = await self._run_tool(step.tool, request)
            # OBSERVE
            self.working_memory.append({"tool": step.tool, ...})
        if not decision.recommendation == "approve":
            await workflow.wait_condition(lambda: self.approved)
        return await workflow.execute_activity(issue_refund, ...)
```

Same names, same order. Every step is an Activity, and the pause is a durable wait. Fix 2 adds one check at the top of this loop.



HOW IT WORKS

Durable Execution reconstructs state from recorded history

Durable Executionは記録された履歴から状態を再構築する



Replay, not redo.

やり直しではなく、リプレイ

- Demo: lookups + decision read back → only the refund ran
- Working memory in Workflow state → back too
- Long-term memory → still your store
- Same idempotency key every retry → Stripe makes one refund

HOW IT WORKS

Replay, not redo

やり直しではなく、リプレイ

NEW WORKER

THE DURABLE LOOP · SIMPLIFIED

turn - ▶ Play

```
for turn in range(MAX_TURNS):
    step = await execute_activity(agent_decide_next_step, working_memory)
    if step.action == "decide":
        break
    if step.action == "ask_customer":
        await wait_condition(answered)        # customer_answer Signal
    else:
        self.working_memory.append(await self._run_tool(step.tool))
refund = await execute_activity(issue_refund, step)
```

EVENT HISTORY · TEMPORAL SERVICE

1 Agent turn 1-5

2 customer_answer ×2

3 lookup_order

4 lookup_customer_history

New Worker runs the code from the top. Finished steps return recorded results. Only the next step runs.

新しいワーカーはコードを最初から実行する。完了済みのステップは記録された結果を返し、実際に動くのは次のステップだけ

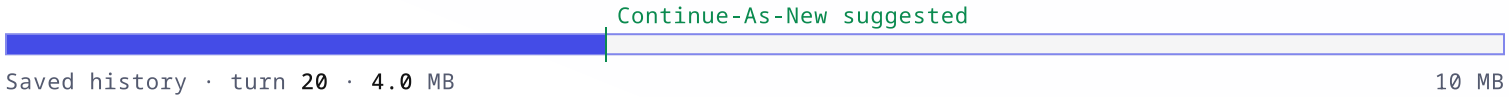
GOTCHA

Every step is saved, so long agent runs build a big history

全ステップが保存されるため、長い実行ほど履歴が大きくなる

Won’t the saved history get too big?

- Temporal saves every step → after a crash, a new Worker replays it to rebuild the agent.
- Every turn re-sends the whole chat to the model → the history grows faster than the chat.
- One result too big (Temporal’s 2 MB limit) → Fix 1: claim check.
- A run that goes on and on (Temporal’s 50 MB limit) → Fix 2: Continue-As-New.



ESTIMATE Example: 20 KB results → turn 20 alone saves 400 KB. About 4 MB total.

FIX 1

Fix 1 · Claim check: save a ticket, not the data

対策1・クレームチェック：データではなく引換券を保存する

Problem: one tool result is too big to save.



Like a coat check: Temporal holds the ticket.

- Needed if the agent pulls a customer's full order history. This demo's results are small.
- Temporal's Python SDK can do this for you (preview).

It shrinks the history, not the prompt.

FIX 2

Fix 2 · Continue-As-New: start a fresh history, same Workflow

対策2・Continue-As-New：同じWorkflowで新しい履歴から始める

Problem: the run goes on so long the history piles up.

Run 1

Workflow ID refund-1234
Run ID a1b2… (one per run)

Run 1 ends · ContinuedAsNew

→

YOU CARRY OVER
order 1234 · $80
lookup ✓ · policy ✓
2 answers
next: issue_refund

Run 2

Workflow ID same
Run ID new · c3d4…

Started · next: issue_refund

Like a new bank statement: it opens with your balance, not every past transaction.

> Carry the state exactly. Summarize the memory.

FIX 2 · IN CODE

Continue-As-New in code (sketch)

Continue-As-Newのコード例（スケッチ）

SKETCH · NOT RUN IN THE DEMO

The loop from slide 16 → one check at the top of each turn

```
for turn in range(MAX_TURNS):                                          ← same loop as slide 16
    if workflow.info().is_continue_as_new_suggested():                 ← history is getting big
        await workflow.wait_condition(workflow.all_handlers_finished)  ← save any answer still arriving
        bookmark = self.where_the_work_stands()                        ← steps done · answers · next step · refund key
        workflow.continue_as_new(replace(request, resume=bookmark))    ← same Workflow ID, fresh history
    step = await workflow.execute_activity(agent_decide_next_step, ...)  ← Run 2 picks up here
```

RUN 1

Why a refund? damaged → Order 1234 paid? yes → Qualifies? yes → ContinuedAsNew

→ bookmark: answers · paid · qualifies · next: refund

RUN 2

Started from bookmark → Next step? issue refund → Completed

Saved history: 3,461 → 554 bytes

The refund's duplicate-check key uses the Run ID → carry it, or roll over before the refund

THE TRADE-OFF

Not every agent needs Durable Execution

すべてのエージェントにDurable Executionが必要なわけではない

YOU MIGHT NOT
不要な場合

- short loop
- the user is right there
- a lost run means asking again
- your runtime already resumes runs

IT EARNS ITS PLACE
必要な場合

- work outlives a process
- long waits on people or systems
- retries against real side effects
- deploys mid-task

Its costs: history holds customer data, so plan retention and encryption. Someone runs the service, or pays for Cloud.

> Temporal won't make the model's choice right → that's evals + guardrails.



COST TO RUN

A restart pays for every model call again

最初からやり直すと、モデル呼び出しの料金をもう一度払う

MEASURED OCT 2 · ONE PASS	4 model calls · 2,962 tokens · $0.0009 (under a tenth of a cent)
Without Temporal Temporalなし INTERRUPTED, START OVER	Same 4 calls again: 2,972 tokens, $0.0009 more
With Temporal Temporalあり INTERRUPTED, RESUME	0 calls re-sent. Finished calls replay from Event History.
AT SCALE ESTIMATE	An agent serving 1M requests a month: ≈3B tokens, ≈$900 in model calls. Every request that starts over pays its share again. 月100万リクエストなら約900ドル。やり直すたびに、その分をもう一度払う

A call in flight when the process dies can be billed again.
List prices: Sep 29

THE DISTINCTION

REASONING
推論

Context
コンテキスト
What the agent can see right now

Long-term memory
長期メモリ
What the agent remembers or looks up

THE OPERATION
処理

Execution state
実行状態
Where the work stands
TEMPORAL

Effect state
副作用の状態
Whether money moved
STRIPE

Memory helps reasoning continue. Temporal helps the operation continue.

メモリは推論を続けさせ、Temporalは処理を続けさせる

TRY IT

Run it yourself

自分で動かしてみよう

```
$ make setup && make run
```

no keys · needs uv + the Temporal CLI

REPO

github.com/temporal-community/temporal-ai-agent-memory-state-stripe

RUN TEMPORAL LOCALLY

```
$ brew install temporal
$ temporal server start-dev
```

TEACH YOUR CODING AGENT TEMPORAL

```
$ npx skills add temporalio/skill-temporal-developer
```

Temporal docs
docs.temporal.io

Payloads too big: External Storage
docs.temporal.io/develop/python/data-handling/external-storage

More in the Temporal & AI Series
Durable human-in-the-loop agents, multi-agent handoffs

History too long: Continue-As-New
docs.temporal.io/develop/python/continue-as-new

Stripe idempotent requests
docs.stripe.com/api/idempotent_requests

TAKEAWAYS

Temporal delivers Durable Execution

TemporalはDurable Executionを提供する

TEMPORAL CUSTOMER RESULTS

Faster velocity

開発スピード向上

Deliver new features ~10x faster.

NEW FEATURES

60–80 days → 5–10 days

MAERSK

Improved reliability

信頼性の向上

Fewer errors, less lost revenue.

PAYMENTS

99.4% → 99.99%

LARGE FINANCIAL INSTITUTION

Full observability

完全な可観測性

See every step of systems and agents.

DEPLOYS FAILED BY CLOUD ERRORS

4% → 0.0001%

NETFLIX

Sources: Maersk and Netflix public case studies · payments figure from a large financial institution



TAKEAWAYS

For agents, it pays off twice

エージェントでは、その効果が二重に得られる

COST

Reduces model calls and costs

モデル呼び出しとコストを削減

AUTONOMY

Enables longer-running workflows and loops

より長いワークフローとループが可能に